

作业一实践报告：LLM 最小代码实践

一、实验目的

本次作业的目标是运行一个最小字符级语言模型，理解大语言模型训练中的基本闭环：文本数据经过 token 或字符编号后输入模型，模型预测下一个 token，计算预测结果和真实目标之间的 loss，再通过反向传播更新参数，最后观察训练前后生成文本和 loss 的变化。

由于当前环境没有教师额外提供的 notebook，也没有安装 PyTorch，本次实验使用 `numpy` 实现了一个最小字符级语言模型。虽然模型规模很小，但它完整包含了字符编号、next-token prediction、交叉熵 loss、梯度计算、参数更新和文本生成过程，能够满足理解 LLM 训练闭环的要求。

二、代码说明

实验代码文件为 `tiny_char_lm.py`。训练文本围绕人工智能辅助网络工程学习、子网划分、CIDR、token、loss 等内容编写。程序首先统计训练文本中出现过的所有字符，建立字符到编号的映射表。每一个汉字、字母、数字、空格或标点都被看作一个 token。

模型采用最小的字符级 bigram 语言模型。它根据当前字符预测下一个字符。例如，在训练文本中如果经常出现“网”后面接“络”，模型经过训练后就会提高“络”作为下一个字符的概率。训练过程中使用交叉熵 loss 衡量预测分布和真实下一个字符之间的差距，并通过梯度下降更新参数。

三、运行记录

第一次运行：基线参数

运行参数如下：

```
batch_size = 32
max_iters = 500
learning_rate = 1.0
```

运行结果：

```
字符表大小 vocab_size = 140
训练文本长度 text_length = 321
训练前 loss: 4.9425
训练后 loss: 2.7345
```

训练前生成文本片段：

率最每中预掩需n地划的用户.s4能低M助9预准辅每搬工汉汉
说变助地汉可可6型k要分重每的都习搬1I变每缀代用个

训练后生成文本片段:

当念M标0会学言需需要低2出出率助计地址生成代汉据L字观不子分接面准缀缀人用验释是CIR整在测确t训
At程

loss 变化记录:

```
| step | loss |  
|---:|---:|  
| 1 | 4.9355 |  
| 100 | 4.3570 |  
| 200 | 3.8521 |  
| 300 | 3.4105 |  
| 400 | 3.0453 |  
| 500 | 2.7345 |
```

从结果看, loss 从 4.9425 降到 2.7345, 说明模型已经学习到训练文本中部分字符之间的连接关系。生成文本虽然还不通顺, 但开始出现“地址”“生成”“验证”“子”等与训练文本相关的字符组合。

第二次运行: 修改参数

为了完成参数修改与对比, 我修改了三个参数:

batch_size 从 32 改为 64

max_iters 从 500 改为 900

learning_rate 从 1.0 改为 0.8

运行结果:

字符表大小 vocab_size = 140

训练文本长度 text_length = 321

训练前 loss: 4.9425

训练后 loss: 2.1978

训练后生成文本片段:

92., 观可以辅最络级人/不最语字符级语言模型算我1C播解
8M程察出的或明察看.

loss 变化记录:

```
| step | loss |  
|---:|---:|  
| 1 | 4.9370 |  
| 100 | 4.4744 |  
| 200 | 4.0401 |  
| 300 | 3.6609 |  
| 400 | 3.3290 |
```

```
| 500 | 3.0396 |  
| 600 | 2.7896 |  
| 700 | 2.5673 |  
| 800 | 2.3697 |  
| 900 | 2.1978 |
```

第二次运行的训练后 loss 更低，说明增加训练轮数后，模型对训练文本的拟合程度更高。生成文本中出现了“字符级语言模型”等更接近训练语料的短语，但整体仍然不是真正通顺的文章。这符合最小字符级模型的特点：它只能学习非常局部的字符转移关系，不能像大型 Transformer 模型那样理解长距离语义。

四、参数对比分析

```
| 项目 | 基线运行 | 修改参数运行 |  
| --- | ---: | ---: |  
| batch_size | 32 | 64 |  
| max_iters | 500 | 900 |  
| learning_rate | 1.0 | 0.8 |  
| 初始 loss | 4.9425 | 4.9425 |  
| 最终 loss | 2.7345 | 2.1978 |
```

从对比结果看，修改参数后的最终 loss 更低。主要原因是 `max_iters` 增加后，模型进行了更多次参数更新；`batch_size` 增大后，每次更新使用的样本更多，loss 估计相对更稳定；`learning_rate` 略微降低后，训练过程更平稳。由于本实验模型和数据都很小，生成文本不能达到自然语言水平，但 loss 的下降已经能够说明训练过程有效。

五、关键概念解释

1. token

token 是模型处理文本时的基本单位。在大型语言模型中，token 可能是一个汉字、一个英文单词、一个词的一部分或一个标点。在本实验中，为了让模型尽可能简单，我把每一个字符都作为一个 token。例如“网络”两个字会被拆成“网”和“络”两个 token。程序会给每个不同字符分配一个编号，这样文本就可以变成数字序列输入模型。

2. loss

loss 是衡量模型预测错误程度的数值。本实验中，模型看到当前字符后，会输出下一个字符的概率分布。如果真实下一个字符的概率很低，loss 就会比较高；如果模型把较高概率分给了真实字符，loss 就会降低。训练的目标就是通过不断更新参数，让 loss 逐渐变小。

3. next-token prediction

next-token prediction 指的是“根据前面的 token 预测下一个 token”。这是语言模型训练中的核心任务之一。例如训练文本中出现“子网划分”，模型在看到“子”后要学习预测“网”，看到“网”后要学习预测“划”。大型语言模型也是在更复杂的结构和更大规模数据上完成类似任务，只

是它们能利用更长上下文和更多参数来学习更复杂的语言规律。

六、实验反思

通过本次实验，我理解了语言模型训练的基本流程。即使是一个很小的字符级模型，也需要经历数据编码、模型预测、loss 计算、反向传播、参数更新和文本生成这几个步骤。训练前，模型参数是随机的，因此生成文本基本是随机字符；训练后，loss 明显下降，生成文本开始出现训练语料中的局部词语和短语。

同时，本实验也说明最小模型有明显局限。它只根据当前字符预测下一个字符，缺少长上下文理解能力，因此生成文本仍然不通顺。真正的大语言模型会使用更复杂的神经网络结构，例如 Transformer 和 Self-Attention，使模型能够综合更长范围的信息。但从学习角度看，这个最小实验帮助我理解了 LLM 的训练闭环，也让我认识到 loss 下降、参数调整和生成效果之间的关系。

七、提交材料说明

本次作业提交材料包括：

1. `tiny_char_lm.py`：最小字符级语言模型代码。
2. `results/baseline.json`：基线运行结果。
3. `results/modified.json`：修改参数后的运行结果。
4. `results_baseline.txt` 和 `results_modified.txt`：终端运行记录。
5. `运行截图.png`：运行结果截图。
6. `作业一实践报告.md`：本实践报告。